Deliverables and Submission

We review submissions in batch, by email, with **no opportunity to ask you anything**. Every gap in your documentation is a gap we have to guess at, and we will guess unfavourably. Assume the reviewer is competent, has 45 minutes, and has never seen your code.

Acceptance gates

These are pass/fail. A submission that fails a gate cannot be scored, and we will not chase you for a fix.

Gate	Requirement
G1	A **public GitHub repository** we can clone without being granted access.
G2	`git clone <repo> && cd <repo> && docker compose up` brings up the entire stack — backend, frontend, database, anything else — on a machine with only Docker installed. No manual migration step, no hand-editing of config, no separately started services.
G3	The app is **seeded on startup** with a usable synthetic network, so a reviewer sees a working system immediately rather than an empty screen.
G4	A **public URL** where the deployed system is running. Openable in a browser with no account, no invite, no VPN, no API key of ours. Free tiers are fine.
G5	The fault simulator is runnable from that public URL or from one documented command, and injecting a fault visibly produces a localized ticket.
G6	A **5-minute demo video** (Loom, YouTube unlisted, Drive link — anything we can watch) showing a fault injected, detected, localized, ticketed, repaired, and auto-verified. This is your insurance: if your deploy is down when we review, the video is what we score.

On G4: hosting a demo on a free tier that cold-starts is fine, but say so in the README so we wait rather than assume it is broken.

Documents in the repository

Five markdown files, at the repo root. Keep them tight — we would rather read two focused pages than ten padded ones.

`README.md`

The front door. What it does, the one-command start, the public URL, the demo video link, and a map of the rest of the docs. A reviewer should be able to run your system from this file alone.

`ARCHITECTURE.md`

The technical heart of your submission. Include:

- **A diagram.** Data flow from pole device to operator screen. Mermaid in the markdown, or a committed image — either is fine, hand-drawn and photographed is fine. It must be legible and it must match what you actually built.
- **Data sourcing and ingestion.** How telemetry arrives, how you handle duplicates, out-of-order messages, clock skew, and bursts.
- **Storage and internal model.** Your schema, and how you represent the network topology. Why this representation and not another.
- **The localization algorithm.** Explain it well enough that we could reimplement it. Cover: how you find the fault boundary, how you group symptoms into one incident, how you handle simultaneous faults, how you compute confidence, and **what you do about the 60% of transformers with no recorded pole ordering**. Give its complexity, and its known failure cases.
- **Noise handling.** Dead sensors versus real outages. Scheduled outages. Debouncing. What your false-positive story is.
- **API surface.** Every endpoint, its method, path, purpose, and shape. A table is fine; OpenAPI is better if it is generated rather than hand-maintained.
- **UI reasoning.** What the operator sees first, and why. What you deliberately did not put on screen. Which decision you expect to be wrong.
- **The AI feature.** What it is, why that spot and not elsewhere, what it costs per call, and what happens when the model is unavailable or wrong.

DEPLOYMENT.md

Written for someone who has your repo and nothing else.

- Prerequisites with versions.
- Exact commands, in order, copy-pasteable.
- Every environment variable: name, what it does, whether it is required, a safe default. Commit a `.env.example`.
- How to verify it worked — what URL to open, what you should see.
- **A troubleshooting section.** This is not optional and it is not filler. List the failure modes you actually hit while building and deploying: port conflicts, migrations racing the database, ARM versus x86 image problems, memory limits on free tiers, CORS, WebSocket upgrades behind a proxy, cold-start timeouts. For each: the symptom you would see, and the fix.
- How to reset to a clean state.

We weight this heavily, and it is not busywork. It is the closest proxy we have for whether you can hand work to someone else.

DECISIONS.md

A log, newest first. For each meaningful decision: what you chose, what you rejected, and why. Include the assumptions you made where the brief was ambiguous — an assumption written down is treated as correct even where we would have chosen differently. End with what you would do with two more weeks, and what you know is currently wrong or fragile.

AI-WORKFLOW.md

How you actually worked.

- Which AI tools, for what.
- What you delegated wholesale versus wrote yourself, and why you drew the line there.
- Two or three concrete cases where the AI was wrong, misleading, or confidently produced something you had to throw away — and how you caught it.
- Roughly how much of the final code is AI-generated. An honest estimate; we are not scoring this number.
- The prompts or session excerpts you consider your best work.

We are not testing whether you use AI. We are testing whether you can tell good AI output from bad, and whether you understand what shipped. **Expect us to pick a function in your repo and ask you to explain it line by line.**

Code expectations

Not a production system, but not a hackathon demo either.

- **Tests where they matter.** We are looking for tests on the localization logic specifically — that is where correctness lives. Broad coverage of controllers and components is not what we want. If you test one thing, test that a known fault in a known topology produces the expected span.
- **Real commit history.** Incremental commits with meaningful messages. A single "initial commit" containing everything tells us nothing about how you work and reads as though the repo was assembled elsewhere.
- **No secrets in the repo.** If you commit a key, we will notice, and it counts against you regardless of whether it was live.
- Consistent formatting, and a linter you actually run.

How to submit

Reply to the email you received this from, before the deadline stated there, with:

1. **GitHub repo URL** (public)
2. **Live public URL**
3. **Demo video link**
4. **A short note, under 300 words**, in the email body: what works, what doesn't, what you cut and why, and the one thing you would fix first. Being straight with us here is a positive signal, not a confession.

Then stop. Do not push commits after the deadline — we review the state of the default branch at the deadline timestamp, and later commits are ignored.

Self-check before you send

- ☐ Cloned my own repo into a fresh directory and ran `docker compose up`. It worked.
- ☐ Opened my public URL in a private browsing window. It worked, with no login.

- [ ] Injected a span fault. Got exactly one ticket, correctly located, with a PIN code.
- [ ] Injected three simultaneous faults. Got three tickets, not one and not thirty.
- [ ] Killed a device's telemetry with power still on. Did **not** get a fault ticket.
- [ ] Ran a scheduled outage. Did **not** get a fault ticket.
- [ ] Repaired a fault. Ticket auto-verified from telemetry without me clicking "resolved".
- [ ] Marked a ticket resolved while the poles were still dark. The system pushed back.
- [ ] All five documents present, and the architecture diagram matches the code I shipped.
- [ ] A stranger could follow `DEPLOYMENT.md` without messaging me.
- [ ] No secrets in git history.
- [ ] I can explain every file in this repo.

Rubric and weights: `04-evaluation.md` (04-evaluation.md).